

03 — SIMD (Single Instruction, Multiple Data)

Pré-requisito: [02-ilp-and-superscalar.md](#)

1. Intuição: uma instrução, vários dados de uma vez

Os dois arquivos anteriores mostraram formas de rodar **instruções diferentes** ao mesmo tempo (pipelining sobrepõe etapas; superescalar despacha múltiplas instruções por ciclo). SIMD é um terceiro tipo de paralelismo, mais direto ainda: **uma única instrução opera sobre vários números ao mesmo tempo**.

Instrução escalar normal:

```
ADD R1, R2, R3 ; um número + um número = um número
```

Instrução SIMD (vetorizada):

```
VADD V1, V2, V3 ; oito números + oito números = oito números,
                ; tudo na MESMA instrução, no MESMO ciclo
```

2. Como isso é fisicamente possível

O chip tem registradores especiais, largos (256 bits, 512 bits, dependendo da geração — chamados de registradores vetoriais), que guardam **vários números pequenos empacotados juntos** em vez de um número só. Um registrador de 256 bits guarda, por exemplo, 8 floats de 32 bits de uma vez. A ULA vetorial faz a mesma operação (soma, multiplicação) nas 8 posições **simultaneamente**, porque o circuito é literalmente replicado 8 vezes lado a lado, todos recebendo o mesmo comando de controle.

Registrador vetorial de 256 bits, guardando 8 floats:

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| float0 | float1 | float2 | float3 | float4 | float5 | float6 | float7 |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

VADD soma dois desses registradores, posição a posição, numa instrução só:

```
A: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]
+ B: [10, 10, 10, 10, 10, 10, 10, 10 ]
= C: [11, 12, 13, 14, 15, 16, 17, 18 ]
```

3. A ligação direta com a conversa de hoje sobre Python vs C++/NumPy

Lembra de quando discutimos por que um loop Python puro é lento e por que NumPy é rápido? **Isso aqui é uma parte real do mecanismo.** Quando você faz `numpy_array * 10`, a implementação em C por trás do NumPy usa instruções SIMD pra processar vários elementos do array numa instrução só — não é só "evitar o overhead do interpretador Python", é literalmente aproveitar hardware que processa múltiplos números em paralelo, de verdade, dentro de um único núcleo de CPU.

Um loop Python elemento-por-elemento nunca aciona SIMD — cada iteração vira uma sequência inteira de operações de interpretador, uma de cada vez. Um loop em C++ bem escrito, que o compilador consegue **auto-vetorizar**, vira instruções SIMD automaticamente.

4. Auto-vetorização: o compilador faz isso por você (às vezes)

```
void soma_arrays(float* a, float* b, float* out, int n) {  
    for (int i = 0; i < n; i++) {  
        out[i] = a[i] + b[i];  
    }  
}
```

Com otimização ligada (-O2/-O3, lembra da regra do Módulo 07 — nunca meça em -O0), o compilador **pode** transformar esse loop pra processar 8 floats por iteração, usando instruções SIMD, sozinho, sem você escrever nada especial. Isso se chama **auto-vetorização**.

Mas isso não é garantido. O compilador só consegue vetorizar quando consegue **provar** que é seguro — por exemplo, que `a`, `b` e `out` não se sobrepõem na memória (lembra do `restrict` mencionado no seu roadmap de Phase 0?). Se houver qualquer possibilidade de sobreposição (aliasing), o compilador desiste da vetorização por segurança, mesmo que na prática não haja problema.

PERGUNTA RÁPIDA 1: por que a palavra-chave `restrict` ajuda o compilador a vetorizar mais agressivamente — e por que `const`, sozinho, **não** resolve o mesmo problema?

► Resposta

Cuidado com uma confusão comum aqui (`const` e `restrict` resolvem problemas diferentes, não o mesmo problema em graus diferentes):

`const float* a` diz ao compilador só isto: *"através deste pointer específico, eu nunca vou escrever"*. Isso **não** diz nada sobre se `a` aponta pra uma região de memória que **outro** pointer (`out`, por exemplo) também alcança. Dois pointers `const` podem perfeitamente apontar pro mesmo endereço um do outro; e um pointer `const` pode continuar sobrepondo memória com um pointer não-`const` que escreve — o compilador não tem garantia nenhuma de que `out[i] = a[i] + b[i]` não está, secretamente, escrevendo em cima de um `a[j]` futuro que ainda vai ser lido. Essa incerteza (chamada **aliasing**) é o que trava a vetorização — não a possibilidade de escrita em si.

`restrict` (ou `__restrict` em muitos compiladores) resolve isso diretamente: é uma **promessa explícita** de que aquele pointer **não** aponta pra memória alcançável por nenhum outro pointer no mesmo escopo. Com essa promessa, o compilador não precisa mais se preocupar que uma escrita através de `out` invalide uma leitura futura através de `a` — ele pode vetorizar livremente.

Resumindo: `const` é sobre **quem pode escrever através deste pointer**. `restrict` é sobre **se este pointer compete por memória com outros pointers**. São perguntas independentes — usar `const` sozinho não dá ao compilador a garantia de não-aliasing que ele precisa pra vetorizar com confiança total; muitas vezes ele ainda vai gerar um "loop de segurança" (runtime check) ou desistir da vetorização se não houver `restrict` também.

5. Largura de vetor: por que "quantos de uma vez" varia

Diferentes gerações/famílias de instrução SIMD em x86 têm larguras diferentes:

Conjunto de instruções	Largura do registrador	Floats de 32 bits por instrução
SSE	128 bits	4
AVX / AVX2	256 bits	8
AVX-512	512 bits	16

Quanto mais largo, mais dado processado por instrução — mas nem toda CPU suporta as larguras mais novas, e usar uma instrução que o hardware não suporta trava o programa (illegal instruction). É por isso que compiladores têm flags de arquitetura-alvo, e por que bibliotecas numéricas sérias frequentemente detectam a CPU em tempo de execução e escolhem qual versão vetorizada usar.

6. Alignment: por que a posição do dado na memória também importa pra SIMD

Um registrador vetorial de 256 bits (8 floats) lê/escreve, idealmente, num endereço que é **múltiplo do tamanho do registrador** (32 bytes, pra 256 bits). Isso não é coincidência — é o mesmo princípio de alignment que você já estudou no Módulo 08 (Phase 0) pra structs, só que agora aplicado a instrução SIMD:

Endereço alinhado (múltiplo de 32):

0x1000: [f0 f1 f2 f3 f4 f5 f6 f7]
^-- 1 transação de memória
busca os 8 de uma vez

Endereço desalinhado:

0x1004: [f0 f1 f2 f3 f4 f5 f6 f7]
^-- pode precisar de 2 transações,
porque os 8 floats cruzam a
fronteira de duas linhas

Um load vetorizado num endereço desalinhado pode custar **mais de uma transação de memória** pra buscar o mesmo dado — o hardware precisa "juntar" pedaços de duas regiões diferentes. Alguns conjuntos de instrução mais antigos nem permitiam load desalinhado (travava o programa); os modernos permitem, mas com penalidade de performance, silenciosa — seu código continua funcionando, só mais devagar, sem aviso nenhum.

PERGUNTA RÁPIDA 2: por que `alignas(32)` (ou `alignas(16)`), que você já viu no Módulo 08 do Phase 0) num array/struct que vai ser processado com SIMD é uma otimização real, não só detalhe estético?

► Resposta

Porque garante, de antemão, que os endereços onde as instruções vetorizadas vão ler/escrever caem exatamente nas fronteiras que o hardware prefere — eliminando a chance de um load vetorizado precisar de transações extras por causa de desalinhamento. É a mesma lógica do `float4` do CUDA que você já viu de relance: 16 bytes, alinhado a 16 bytes, uma transação só.

7. Por que isso importa para CUDA

Aqui está a conexão mais direta e importante do módulo inteiro: **uma GPU é, em espírito, uma máquina SIMD levada ao extremo**. Quando uma GPU roda um warp (32 threads), ela está literalmente executando **a mesma instrução, ao mesmo tempo, sobre 32 dados diferentes** — o mesmo princípio do **VADD** de 8 floats aqui, só que com 32 "pistas" em vez de 8, e cada "pista" é chamada de thread em vez de posição de registrador vetorial.

Essa é a razão de **divergência de warp** (quando threads dentro do mesmo warp tomam caminhos diferentes num **if**) ser cara: assim como uma instrução SIMD de CPU não pode fazer "soma nas posições 0-3 e multiplicação nas posições 4-7" numa instrução só, um warp não consegue rodar dois caminhos de código diferentes simultaneamente — ele executa os dois caminhos em sequência, desperdiçando parte da largura.

8. Erros comuns

- Achar que SIMD é "múltiplos núcleos". Não é — é paralelismo **dentro de um único núcleo**, numa única instrução. Combina com múltiplos núcleos (cada núcleo pode ter sua própria unidade SIMD), mas são conceitos independentes.
- Assumir que todo loop vetoriza automaticamente. Loops com dependência entre iterações, aliasing não resolvido, ou padrões de acesso complexos frequentemente não vetorizam — vale inspecionar (**-S**, do Módulo 07, ou **-Xptxas -v** que aparece mais tarde em CUDA) em vez de assumir.
- Confundir SIMD com paralelismo de dado em geral. SIMD é uma forma específica, com uma instrução controlando várias lanes simultaneamente — diferente de threads independentes, que têm seu próprio fluxo de controle cada uma.

PREDICT BEFORE RUNNING (conceitual)

```
void f(float* a, float* b, float* out, int n) {
    for (int i = 0; i < n; i++) {
        out[i] = a[i] + b[i];
    }
}

void g(const float* __restrict a, const float* __restrict b, float* __restrict
out, int n) {
    for (int i = 0; i < n; i++) {
        out[i] = a[i] + b[i];
    }
}
```

- As duas funções calculam exatamente a mesma coisa. Qual delas o compilador tem mais chance de vetorizar agressivamente, e por quê?
- Se você chamasse **f(x, x+1, x, 100)** (arrays sobrepostos de propósito), o resultado seria diferente do que chamar com arrays realmente separados?

► Explicação

- g**, por causa do **__restrict** (e do **const** nos dois ponteiros de entrada) — o compilador tem garantia explícita de que não há aliasing, então pode vetorizar sem se preocupar em provar isso sozinho.

2. Sim, potencialmente — com arrays sobrepostos, o resultado de uma versão vetorizada (processando vários elementos "ao mesmo tempo") pode diferir de uma versão puramente sequencial, porque a ordem em que leituras e escritas acontecem muda. É exatamente por isso que `restrict` é uma **promessa** que você faz ao compilador — se for mentira (arrays realmente se sobrepõem), o comportamento vira indefinido, no mesmo espírito do Módulo 06 de undefined behavior que você já domina.